IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts Science Commerce and Management

High Level Design

on

Identity Security Framework for Energy

Student Name:Shalaya P B

Industry Mentor:
Shashank Koram

Guided by: Suzana Sharol Prabhakar

Table of Contents

1. Introduction. 3
   1.1. Scope of the document. 3
   1.2. Intended Audience.... 3
   1.3. System overview.... 3

2. System Design. 4
   2.1. Application Design....4
   2.2. Process Flow.... 4
   2.3. Information Flow....6
   2.4. Components Design 6
   2.5. Key Design Considerations 7
   2.6. API Catalogue. 7

3. Data Design....8
   3.1. Data Model.... 8
   3.2. Data Access Mechanism....8
   3.3. Data Retention Policies 8
   3.4. Data Migration 8

4. Interfaces....8

5. State and Session Management....9

6. Caching 9

7. Non-Functional Requirements 9
   7.1. Security Aspects....9
   7.2. Performance Aspects.... 10

8. References....10

1. Introduction

1.1 Scope of the Document

This document explains the big picture of how the EnergyGuard website works. It covers how the website is tied together, how it keeps data secure, and how it displays information. Specifically, it focuses on three main things: how the login checkpoints work, how the system builds the live energy charts, and how user accounts are safely stored in the database.

1.2 Intended Audience

- Technical Leaders: To double-check that the tools we chose (like Python and SQLite) are safe and right for the job.
- The Coding Team: To give developers a bird's-eye view of how the whole website connects together before they start typing actual code.
- Project Managers & Testers: To help them understand the entire journey of a user—from signing up to viewing their dashboard. This helps them plan new features and write better tests.

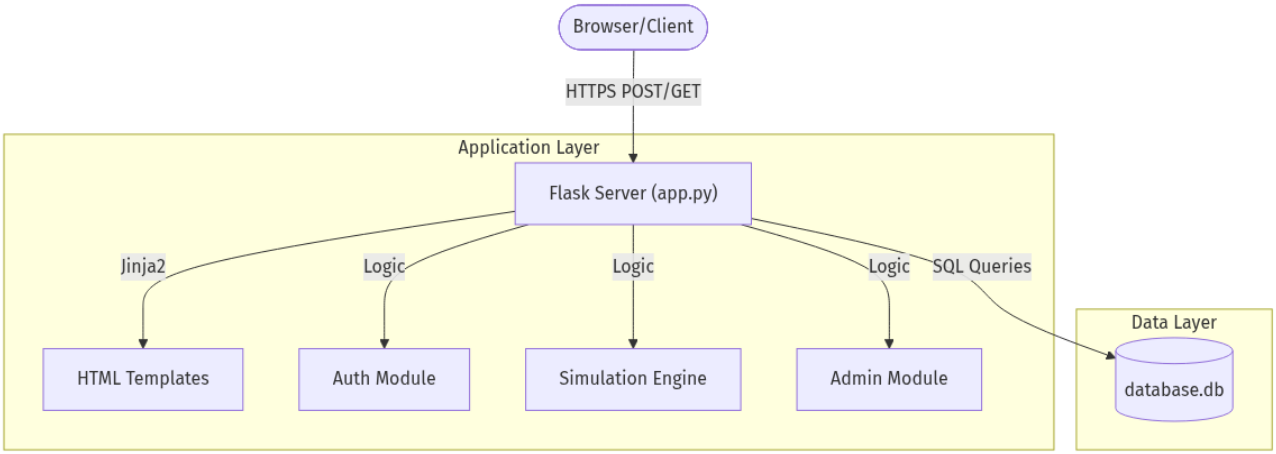
1.3 System Overview

EnergyGuard is essentially a secure locked door that protects energy information. Before anyone can see the data, the website forces them to log in with a secure password. Once logged in, the system shows the user a personal dashboard displaying things like daily electricity usage, solar power saved, and alerts for machines like air conditioners or car chargers.

To keep the platform safe over time, there is a hidden "Admin" role built into the system. This Admin can look at the full list of users and easily delete fake or unwanted accounts, without interrupting the normal users who are just trying to check their energy stats.

2. System Design

2.1 Application Design

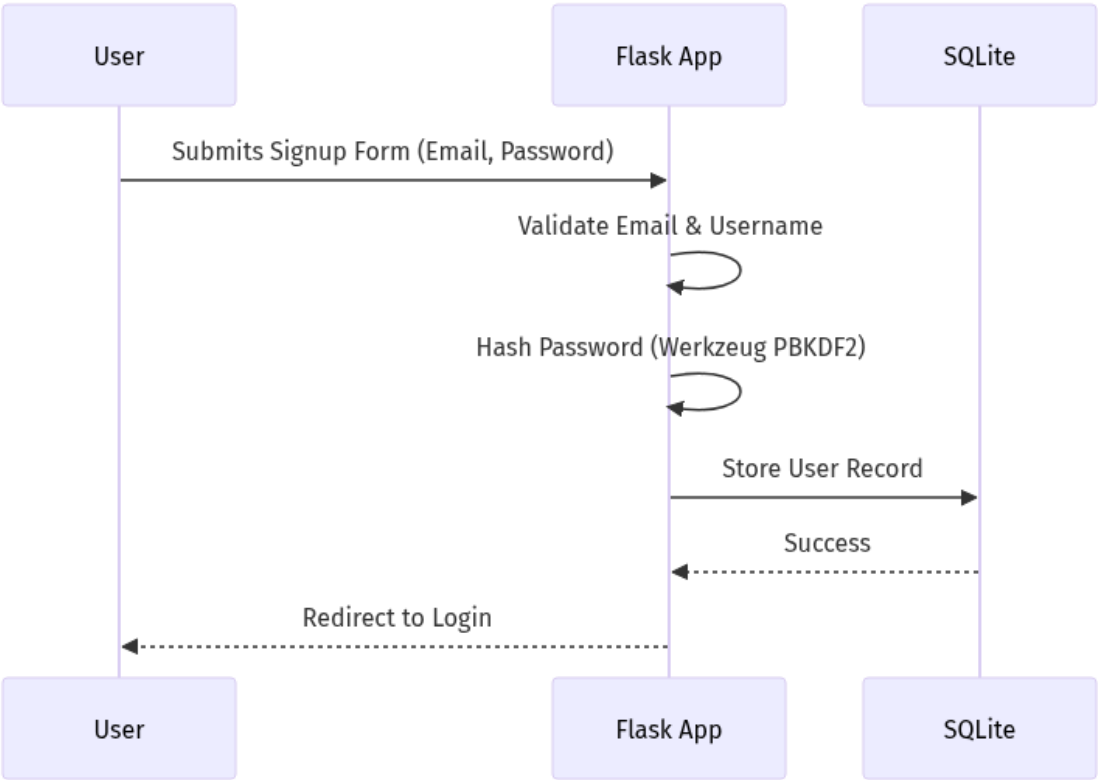


The application is split into three simple pieces that work together smoothly: the user's screen, the brain of the app, and the filing cabinet where data lives. Python acts as the middleman passing notes between what the user clicks and what the database saves.

2.2 Process Flows

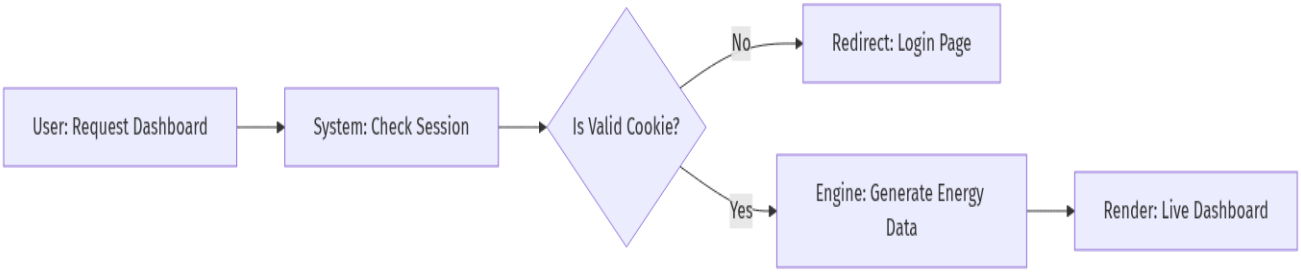
A. Secure Registration and Log In

- Users start by typing their details to either make a new account or log in. The system strictly checks these details. If they are wrong, it bounces the user back with an error. If they are correct, it creates a secure memory (a session cookie) so the user doesn't have to log in on every single page click.



B. Loading the Energy Dashboard

- Once safely inside, the system needs numbers to show. It runs a special math algorithm (using the current day and the user's ID) to create specific, realistic electricity numbers and hardware warnings, sending those numbers directly to the screen.



2.3 Information Flow

Phase 1: User Input

- A person types information into the web page (like signing up) or clicks a link to view the dashboard. That request travels safely from their web browser to our Flask server.

Phase 2: Security & Logic (Behind the Scenes)

- The server double-checks everything. It makes sure emails aren't taken and passwords aren't too short. If it's a password, it mathematically scrambles (hashes) it so it is never readable to anyone, not even the server owner.

Phase 3: Data Saving

- The server uses a tool called SQLAlchemy to quietly save or read this data from the energyguard.db file. This acts as a permanent vault, so information is never lost if the computer restarts.

Phase 4: Real-Time Feedback

- The server finally loads the next webpage. It shows the user friendly, color-coded pop-ups (like "Account Created!" or "Invalid Password") so they never feel confused about what just happened.

2.4 Component Design

- Security Module: Handles creating the password scrambles, remembering who is logged in, and kicking people out who shouldn't be there.
- Energy Data Engine: Because we don't have real wind turbines, this part handles all the math to invent realistic electricity statistics and alerts based on the calendar day.

- Admin Control Panel: The hidden tools that let an administrator easily view the whole user list and permanently delete rule-breakers.

2.5 Key Design Considerations

- Safety Above All: The app assumes anything the user types could be dangerous. Everything is verified and scrubbed clean before it touches the database.
- Smart Storage Links: Instead of writing messy raw SQL code, we use a Python library (SQLAlchemy) which acts as an automatic translator. This keeps the code super clean and blocks hackers easily.
- Instant Speed: Because the database file lives locally right next to the server code, fetching data takes absolutely zero wait time.

2.6 API Catalogue

Web Link	Type	Description	Who Can Access?
/login	GET / POST	Looks at typed details to let users securely into the app.	Public / Anyone
/signup	GET / POST	Creates a brand new account and scrambles the password.	Public / Anyone
/logout	GET	Drops the secure memory (cookie) and logs the person out safely.	Auth Required
/dashboard	GET	The main webpage where live energy numbers, alerts, and graphs appear.	Auth Required
/admin/del..	POST	A strict, hidden tool to permanently erase an account from the system.	Admin Only

3. Data Design

3.1 Data Model

- User Accounts: The database mainly focuses on storing user profiles. For every person, it saves their unique ID, Username, Email, a hidden secret Password, and the time they last logged in.

3.2 Data Access Mechanism

- Reading the Database: Instead of forcing developers to write messy database codes (SQL), the project uses a Python tool called SQLAlchemy. This acts like an automatic translator, making reading and saving data incredibly simple and safe.

3.3 Data Retention Policies

- Saving Accounts: All information is permanently saved inside a local file called energyguard.db.
- Deleting Accounts: A user's data stays there forever until an official Administrator clicks "Delete", which erases that specific profile completely.

3.4 Data Migration

- Automatic Setup: The very first time a developer turns on the application, Python automatically builds all the required database tables. This saves hours of manual setup.

4. Interfaces

➢ User Interface (The Website)

- Look and Feel: A clean, modern website layout designed specifically to highlight energy numbers and hardware warnings.
- Pop-up Alerts: The website uses bright, color-coded message banners to instantly tell the user things like "Account Created!" or "Wrong Password!".

➢ System Interface (Behind the Scenes)

- Local Connections: The website server retrieves data by reading a file that lives directly next to it on the same computer. This means it never has to wait for slow internet database connections.

5. State and Session Management

- Staying Logged In: When a user logs in, the server gives their web browser a secure, hidden "cookie" locked by a secret key. This acts like a VIP badge so the website remembers exactly who they are as they click from page to page.

6. Caching

- Fast Loading Pages: The user's web browser automatically remembers the design colors (style.css) so the website pages load beautifully and instantly.
- Fresh Numbers Only: The daily energy numbers on the dashboard are *never* cached. The server always calculates the exact daily math fresh every single time the user clicks on the dashboard so it's never outdated.

7. Non-Functional Requirements

7.1 Security Aspects

- Password Hiding: Passwords are mathematically scrambled so intensely that even if a hacker stole the database file, they wouldn't be able to read the actual passwords.
- Code Bouncers: Code rules automatically block normal users from accessing secret Admin pages, even if they guess the correct web address.
- Hacker Defense: Using our SQLAlchemy translating tool automatically blocks standard network hackers who try to break the site by typing malicious codes into the login boxes.

7.2 Performance

- Lightning Fast: Because the daily energy numbers are generated locally by a simple math algorithm (instead of downloading them from a massive server), the dashboard page loads in milliseconds.
- Small Footprint: The entire project runs on a tiny database file, making the web application incredibly lightweight and easy to run on almost any computer hardware.

8. References

- Flask Web Framework: https://flask.palletsprojects.com/
- Flask-SQLAlchemy Setup Guide: https://flask-sqlalchemy.palletsprojects.com/